

Lambda Architecture

1. What is Lambda Architecture?

Lambda Architecture is a big-data design pattern where:

None

Fast pipeline + Slow pipeline

run in parallel

update same destination

It combines:

None

Low latency results

High accuracy results

Scalable analytics

2. Core Idea

Use two systems together:

None

Speed Layer → Immediate approximate data

Batch Layer → Accurate recomputed data

Serving Layer → Final query output

3. Why It Exists

Single system often cannot provide both:

None

Real-time speed

AND

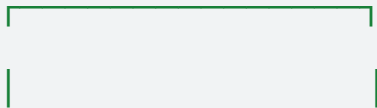
Perfect accuracy at scale

So Lambda splits responsibilities.

4. Architecture Diagram

None

Incoming Events



Speed Layer



Batch Layer



Realtime DB



Hadoop/Spark/HDFS



└─ Merge Results ──> Serving Layer

5. Speed Layer (Fast Pipeline)

Purpose:

None

Immediate updates

Low latency dashboards

Recent counters

Alerts

Uses:

None

Redis

Kafka Streams

Flink

Approximation algorithms

In-memory processing

6. Tradeoffs of Speed Layer

None

May lose data on crash

Approximate counts

Temporary inconsistency

Lower durability

7. Batch Layer (Slow Pipeline)

Purpose:

None

Full historical processing

Recompute exact results

Correct earlier mistakes

Reliable analytics

Uses:

None

Spark

Hive

Hadoop

HDFS

Large SQL warehouses

8. Tradeoffs of Batch Layer

None

High latency

More compute cost

Slower refresh

Heavy infrastructure

9. Example: Website Analytics

User visits page.

Speed Layer instantly updates:

None

Today's views = 10,521

Batch Layer later recalculates:

None

Actual views after dedupe = 10,498

10. Example: E-commerce

None

Realtime inventory estimate

+

Nightly exact stock reconciliation

11. Where Lambda is Useful

Use for:

None

Big data analytics

Fraud monitoring

Recommendation metrics

Search ranking signals

Ad click systems

Operational dashboards

12. Main Benefits

None

Fast user-facing metrics

Accurate final reporting

Handles huge data scale

Fault tolerance

Historical recomputation possible

13. Main Problems

None

Two pipelines to maintain

Duplicate business logic

Higher engineering cost

Complex debugging

Data reconciliation issues

14. Why Teams Moved Away Sometimes

Maintaining both pipelines means:

None

Write code twice

Test twice

Operate twice

This motivated Kappa architecture.

15. What is Kappa Architecture?

Instead of two systems:

None

Single streaming pipeline

Built on append-only log

Usually:

None

Kafka → Stream Processor → Database

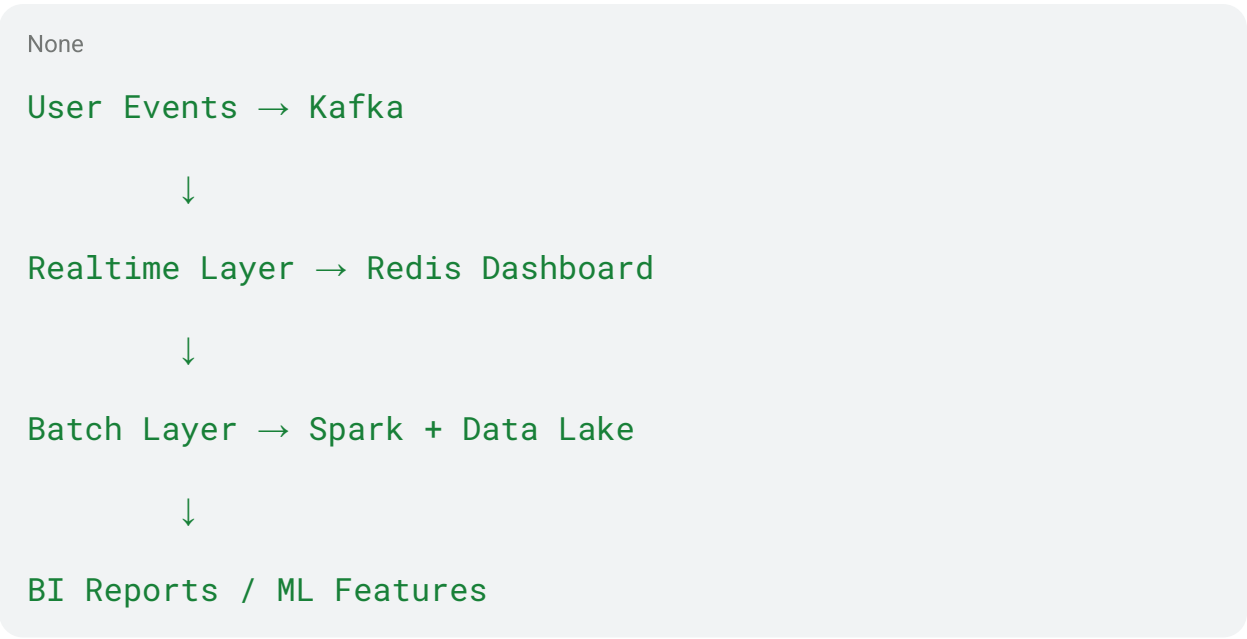
Reprocessing = replay old logs.

16. Lambda vs Kappa

Feature	Lambda	Kappa
Pipelines	Two	One
Complexity	High	Lower
Batch Recompute	Native	Replay logs

Real-time	Yes	Yes
Maintenance	Harder	Easier

17. Best HLD Example



18. Interview Talking Points

If asked analytics system:

None

Use Lambda when both real-time and exact historical correctness matter.

Use speed layer for quick metrics.

Use batch layer for reconciliation.

If simpler ops needed, consider Kappa.

19. When Not to Use Lambda

Avoid if:

None

Small data volume

Small team

No real-time need

Low budget

Simple reporting only

20. One-Line Summary

Lambda Architecture uses parallel fast and slow data pipelines so systems can deliver immediate approximate results now and accurate recomputed results later.